Divide & Conquer



Charles Aunkan Gomes
Lecturer, Dept. of CSE
United International University

Courtesy: Md Shadman Aadeeb

Divide and Conquer

THE FIRST PROBLEM SOLVING STRATEGY

Divide and Conquer

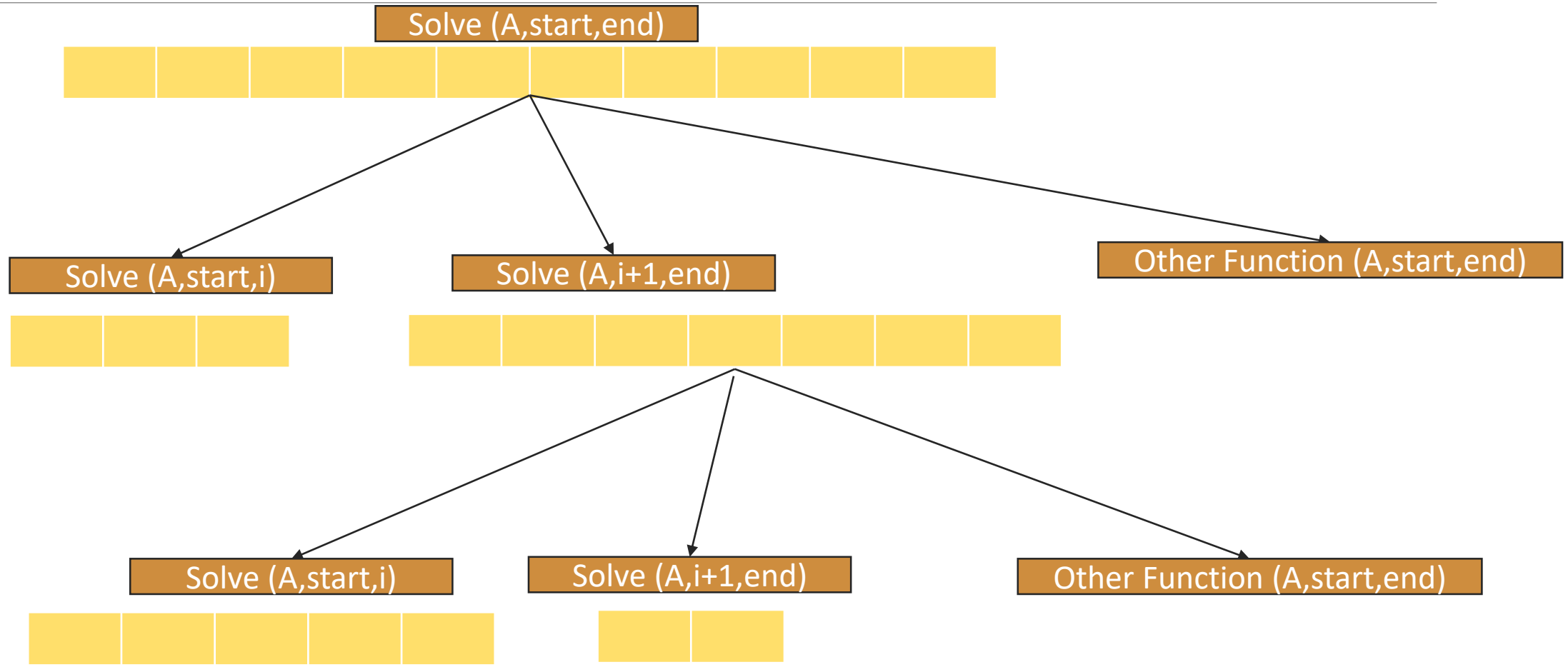
The steps of the Divide and Conquer approach are:

1. **Divide**: Keep splitting the problem into smaller sub-problems.
2. **Conquer**: Solve the smallest sub-problems directly
3. **Combine**: Merge the solutions of the subproblems to produce the final solution for the original problem.

Divide and Conquer

- Note that the divide and conquer problems you will be solving are mostly **based on arrays**
- However, **not all divide and conquer** problems are based on arrays
- Example : **Strassen's algorithm for matrix multiplication (Not in syllabus)**

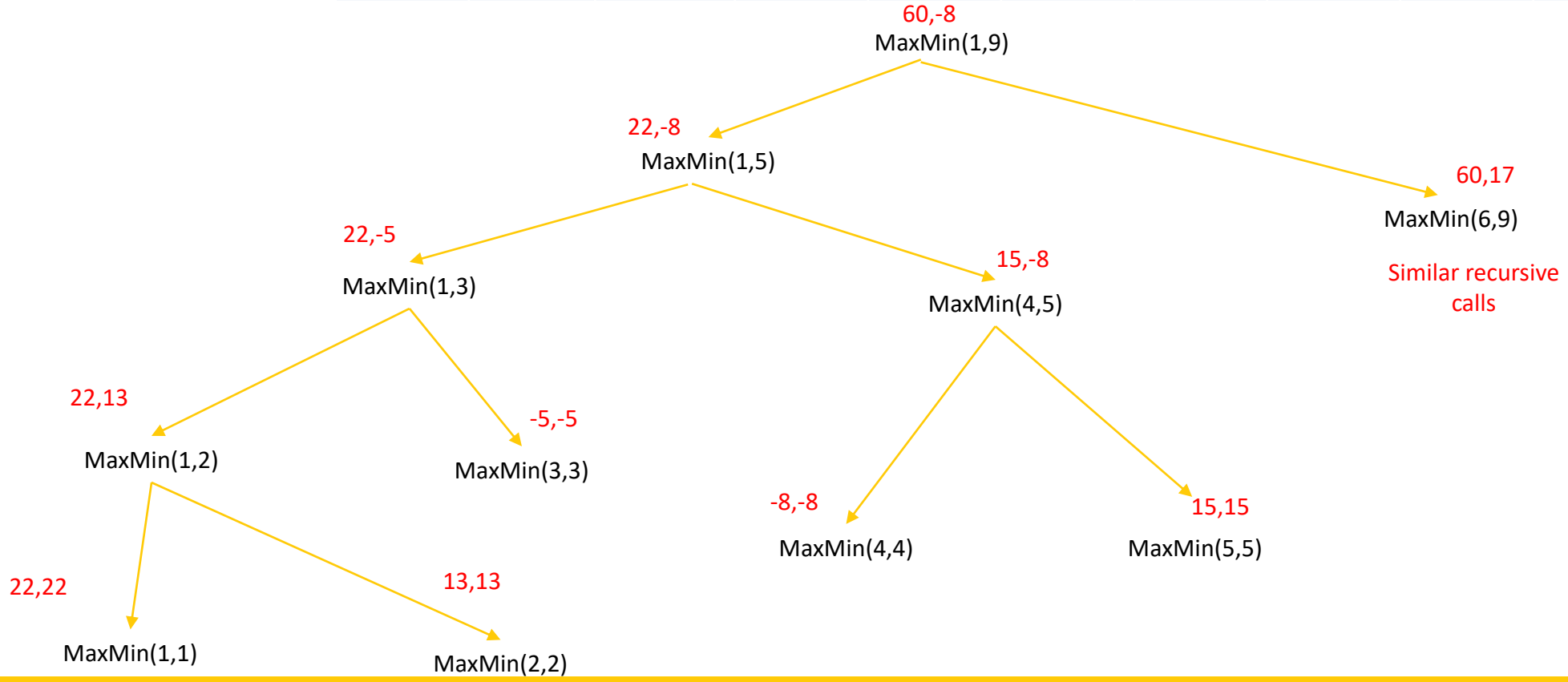
Divide and Conquer (Basic structure for array based problems)



MIN MAX PROBLEM

Finding Maximum and Minimum

Index	1	2	3	4	5	6	7	8	9
Array	22	13	-5	-8	15	60	17	31	47



Pseudocode

```
Find_maxmin (A, i, j)
     if i = j:
            return A[i], A[i]                          // base case
     else if i < j:
            mid = (i+j)/2;                             // divide
            mx1, min1 = Find_maxmin(A, i, mid)         // conquer
            mx2, min2 = Find_maxmin(A, mid+1, j)       // conquer
            fmax, fmin = -1, -1                        // combine
            if mx1 > mx2: fmax = mx1
            else fmax = mx2
            if min1 < min2: fmin = min1
            else fmin = min2
            return fmax, fmin
```

Time Complexity

$$T(n) = \begin{cases} b & \text{if } n = 1 \\ 2T(n/2) + b & \text{if } n > 1 \end{cases}$$

By solving the recurrence, we get

$T(n)$ is $O(\ n\ )$

MAXIMUM SUB ARRAY PROBLEM

MAXIMUM SUB ARRAY PROBLEM

Determine the maximum sub array from this array.

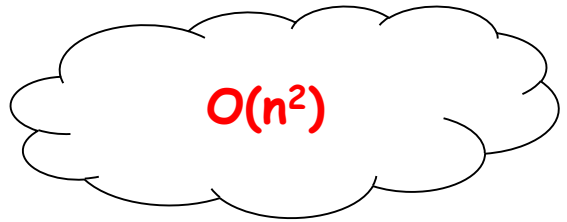
-16	-23	18	20	-7	12	-5
0	1	2	3	4	5	6

Brute Force Approach

All possible contiguous subarrays

- A[1..1], A[1..2], A[1..3], ..., A[1..(*n*-1)], A[1..*n*]
- A[2..2], A[2..3], ..., A[2..(*n*-1)], A[2..*n*]
- ...
- A[(*n*-1)..(*n*-1)], A[(*n*-1)..*n*]
- A[*n*..*n*]

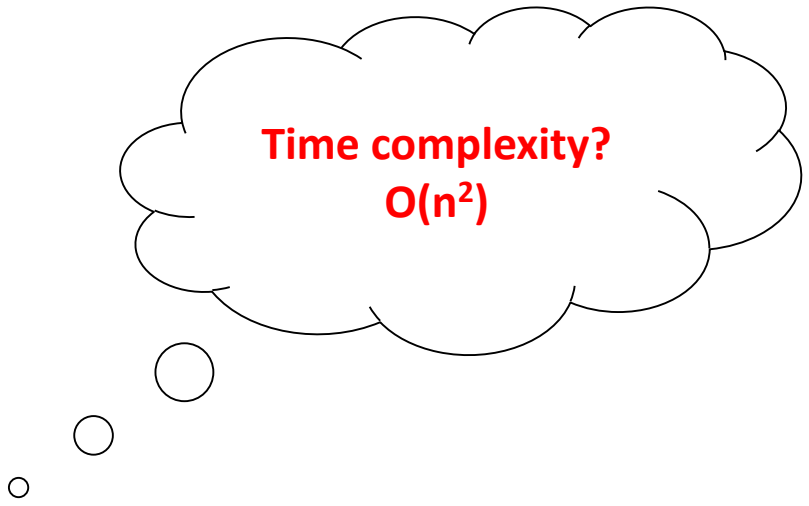
How many of them in total?



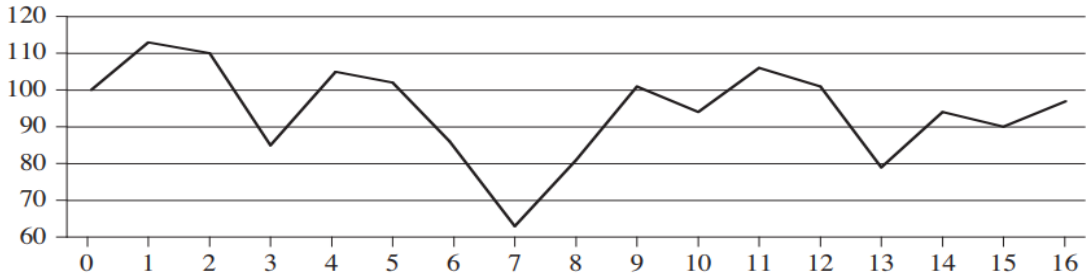
Algorithm: For each subarray, compute the sum.
Find the subarray that has the maximum sum.

Brute Force Approach

```
max = -∞
for i = 1 to n do
begin
    sum = 0
    for j = i to n do
    begin
        sum = sum + A[j]
        if sum > max
        then max = sum
    end
end
```



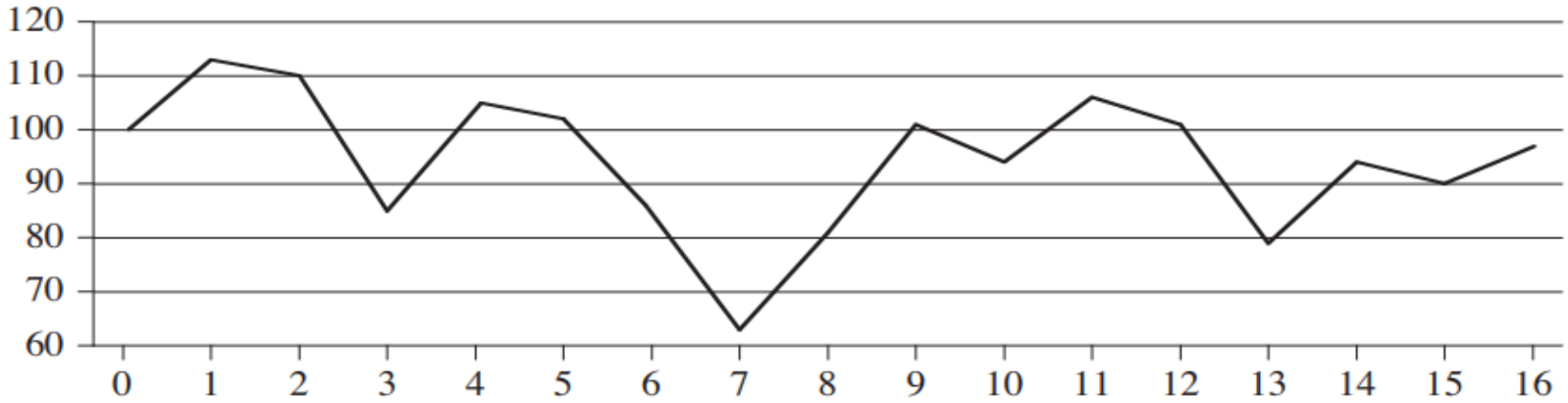
The Maximum Sub Array Problem- practical example



Day	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Price	100	113	110	85	105	102	86	63	81	101	94	106	101	79	94	90	97
Change		13	−3	−25	20	−3	−16	−23	18	20	−7	12	−5	−22	15	−4	7

- This is the **prediction** of prices of share of a company.
- You want to buy the share on the day when the price is the lowest
- You want to sell the share on the day when the price is the highest
- Identify these two days

The Maximum Sub Array Problem- practical example



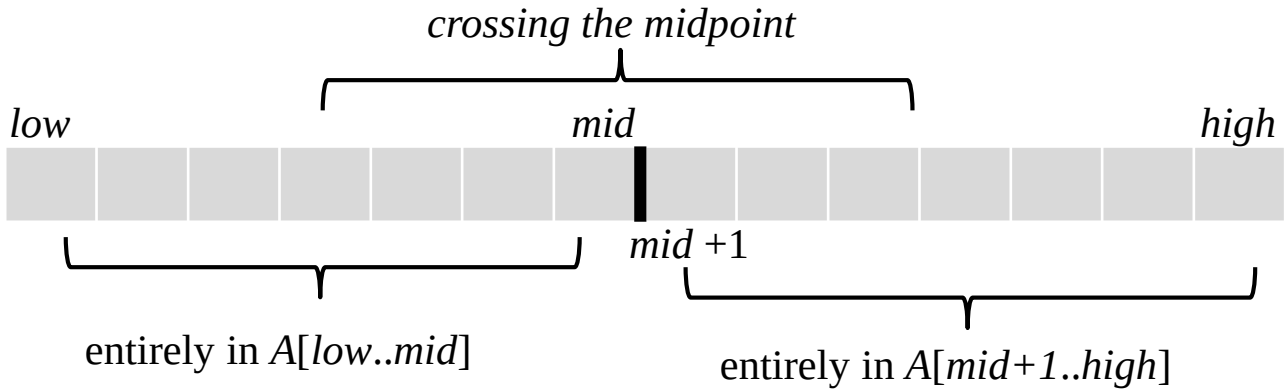
Day	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Price	100	113	110	85	105	102	86	63	81	101	94	106	101	79	94	90	97
Change		13	−3	−25	20	−3	−16	−23	18	20	−7	12	−5	−22	15	−4	7

Buy on the 7th day and sell on the 11th day.
Total profit= 106-63=43
Also Total profit = 18+20-7+12=43

The Divide and Conquer Approach for Max Sub Array

Possible locations of a maximum subarray $A[i..j]$ of $A[low..high]$, where $mid = \lfloor (low + high)/2 \rfloor$

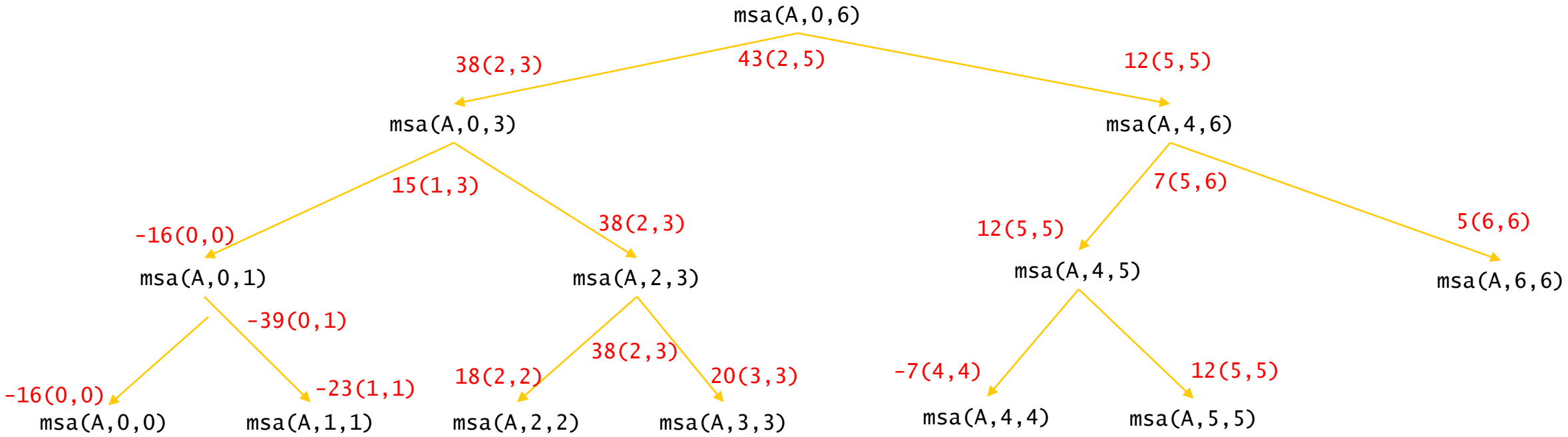
- entirely in $A[low..mid]$ $(low \le i \le j \le mid)$
- entirely in $A[mid+1..high]$ $(mid < i \le j \le high)$
- crossing the midpoint $(low \le i \le mid < j \le high)$



Possible locations of subarrays of A[low..high]

The Divide and Conquer Approach for Max Sub Array

-16	-23	18	20	-7	12	-5
0	1	2	3	4	5	6



Pseudocode

```
FIND-MAXIMUM-SUBARRAY(A, low, high)
 1  if high == low
 2      return (low, high, A[low])          // base case: only one element
 3  else mid = ⌊(low + high)/2⌋
 4      (left-low, left-high, left-sum) =
            FIND-MAXIMUM-SUBARRAY(A, low, mid)
 5      (right-low, right-high, right-sum) =
            FIND-MAXIMUM-SUBARRAY(A, mid + 1, high)
 6      (cross-low, cross-high, cross-sum) =
            FIND-MAX-CROSSING-SUBARRAY(A, low, mid, high)
 7      if left-sum ≥ right-sum and left-sum ≥ cross-sum
 8          return (left-low, left-high, left-sum)
 9      elseif right-sum ≥ left-sum and right-sum ≥ cross-sum
10          return (right-low, right-high, right-sum)
11      else return (cross-low, cross-high, cross-sum)
```

Pseudocode

```
FIND-MAX-CROSSING-SUBARRAY(A, low, mid, high)
 1  left-sum = −∞
 2  sum = 0
 3  for i = mid downto low
 4       sum = sum + A[i]
 5       if sum > left-sum
 6            left-sum = sum
 7            max-left = i
 8  right-sum = −∞
 9  sum = 0
10  for j = mid + 1 to high
11       sum = sum + A[j]
12       if sum > right-sum
13            right-sum = sum
14            max-right = j
15  return (max-left, max-right, left-sum + right-sum)
```

-16	-23	18	20	-7	12	-5
0	1	2	3	4	5	6

Pseudocode

```
FIND-MAX-CROSSING-SUBARRAY(A, low, mid, high)
 1  left-sum = −∞
 2  sum = 0
 3  for i = mid downto low
 4       sum = sum + A[i]
 5       if sum > left-sum
 6            left-sum = sum
 7            max-left = i
 8  right-sum = −∞
 9  sum = 0
10  for j = mid + 1 to high
11       sum = sum + A[j]
12       if sum > right-sum
13            right-sum = sum
14            max-right = j
15  return (max-left, max-right, left-sum + right-sum)
```

leftSum=-inf

-16	-23	18	20	-7	12	-5
0	1	2	3	4	5	6

Pseudocode

```
FIND-MAX-CROSSING-SUBARRAY(A, low, mid, high)
 1  left-sum = −∞
 2  sum = 0
 3  for i = mid downto low
 4       sum = sum + A[i]
 5       if sum > left-sum
 6            left-sum = sum
 7            max-left = i
 8  right-sum = −∞
 9  sum = 0
10  for j = mid + 1 to high
11       sum = sum + A[j]
12       if sum > right-sum
13            right-sum = sum
14            max-right = j
15  return (max-left, max-right, left-sum + right-sum)
```



leftSum=-inf | Sum =0

-16	-23	18	20	-7	12	-5
0	1	2	3	4	5	6

Pseudocode

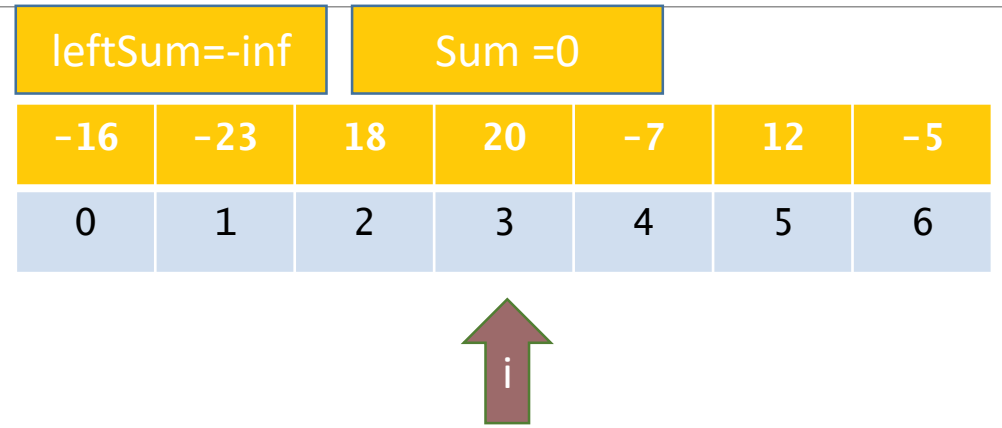
```
FIND-MAX-CROSSING-SUBARRAY(A, low, mid, high)
 1  left-sum = −∞
 2  sum = 0
 3  for i = mid downto low
 4       sum = sum + A[i]
 5       if sum > left-sum
 6            left-sum = sum
 7            max-left = i
 8  right-sum = −∞
 9  sum = 0
10  for j = mid + 1 to high
11       sum = sum + A[j]
12       if sum > right-sum
13            right-sum = sum
14            max-right = j
15  return (max-left, max-right, left-sum + right-sum)
```



leftSum=-inf		Sum =0				
-16	-23	18	20	-7	12	-5
0	1	2	3	4	5	6

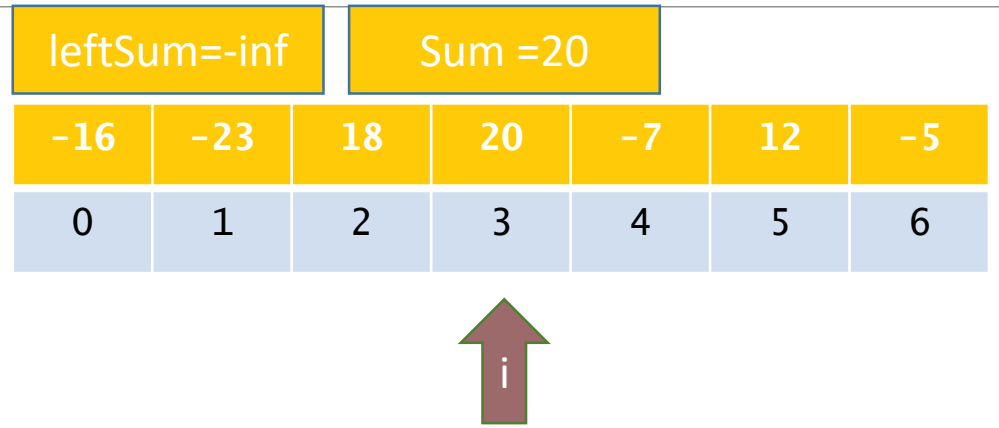
Pseudocode

```
FIND-MAX-CROSSING-SUBARRAY(A, low, mid, high)
 1  left-sum = −∞
 2  sum = 0
 3  for i = mid downto low
 4       sum = sum + A[i]
 5       if sum > left-sum
 6            left-sum = sum
 7            max-left = i
 8  right-sum = −∞
 9  sum = 0
10  for j = mid + 1 to high
11       sum = sum + A[j]
12       if sum > right-sum
13            right-sum = sum
14            max-right = j
15  return (max-left, max-right, left-sum + right-sum)
```



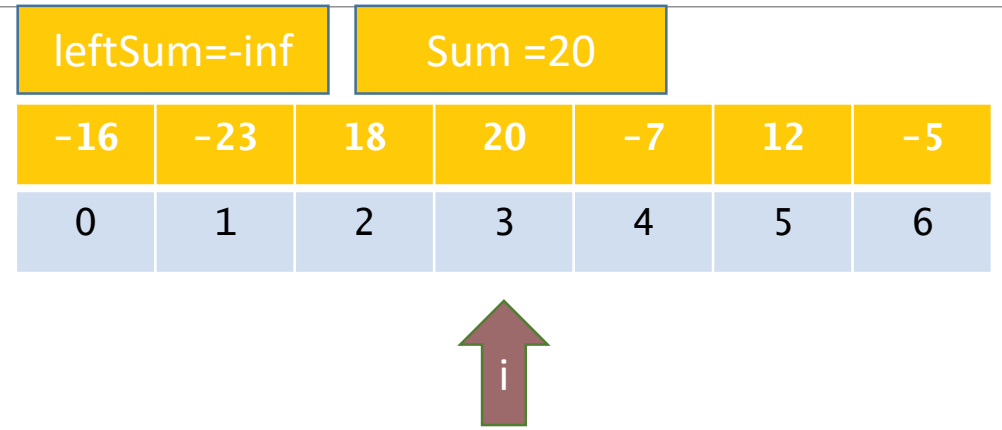
Pseudocode

```
FIND-MAX-CROSSING-SUBARRAY(A, low, mid, high)
 1  left-sum = −∞
 2  sum = 0
 3  for i = mid downto low
 4       sum = sum + A[i]
 5       if sum > left-sum
 6            left-sum = sum
 7            max-left = i
 8  right-sum = −∞
 9  sum = 0
10  for j = mid + 1 to high
11       sum = sum + A[j]
12       if sum > right-sum
13            right-sum = sum
14            max-right = j
15  return (max-left, max-right, left-sum + right-sum)
```



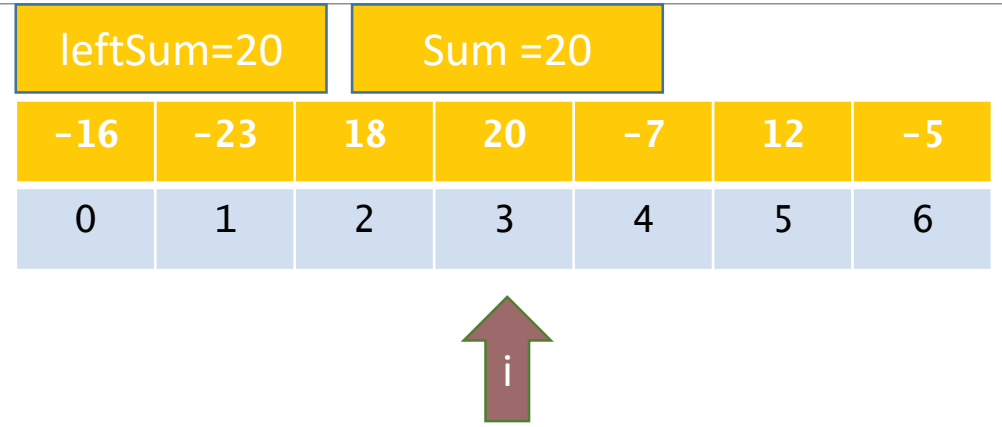
Pseudocode

```
FIND-MAX-CROSSING-SUBARRAY(A, low, mid, high)
 1  left-sum = −∞
 2  sum = 0
 3  for i = mid downto low
 4       sum = sum + A[i]
 5       if sum > left-sum
 6            left-sum = sum
 7            max-left = i
 8  right-sum = −∞
 9  sum = 0
10  for j = mid + 1 to high
11       sum = sum + A[j]
12       if sum > right-sum
13            right-sum = sum
14            max-right = j
15  return (max-left, max-right, left-sum + right-sum)
```



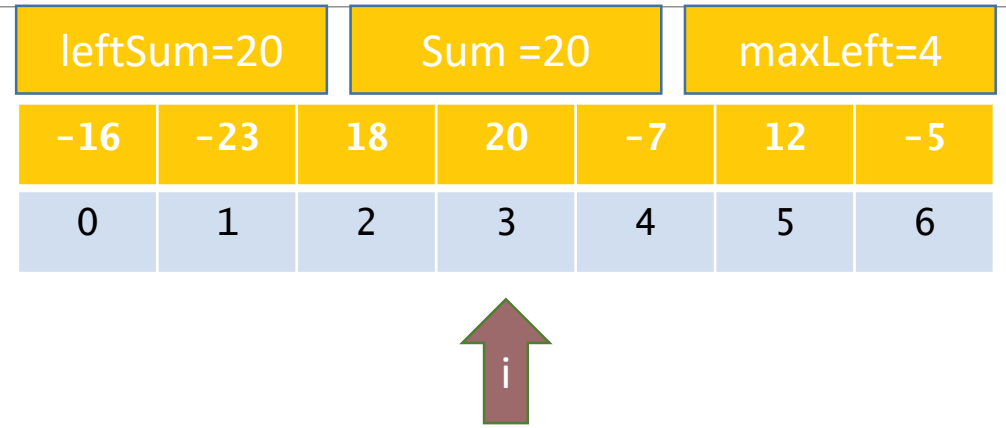
Pseudocode

```
FIND-MAX-CROSSING-SUBARRAY(A, low, mid, high)
 1  left-sum = −∞
 2  sum = 0
 3  for i = mid downto low
 4       sum = sum + A[i]
 5       if sum > left-sum
 6            left-sum = sum
 7            max-left = i
 8  right-sum = −∞
 9  sum = 0
10  for j = mid + 1 to high
11       sum = sum + A[j]
12       if sum > right-sum
13            right-sum = sum
14            max-right = j
15  return (max-left, max-right, left-sum + right-sum)
```



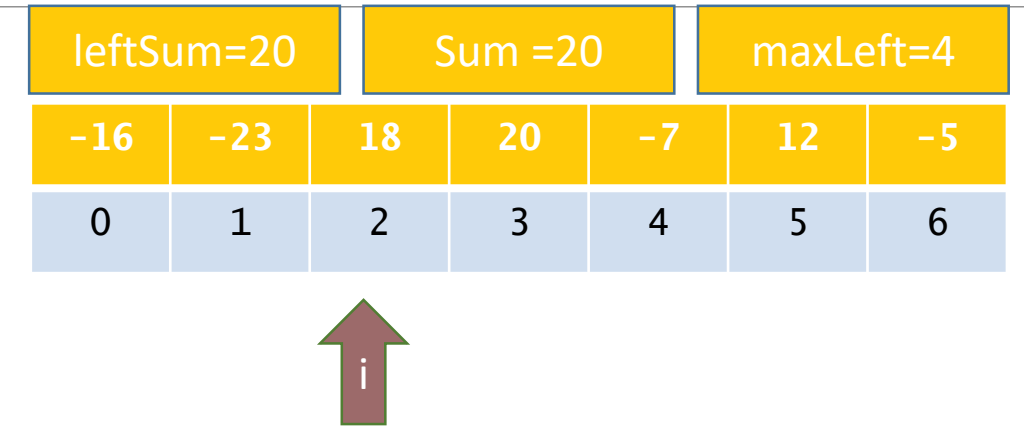
Pseudocode

```
FIND-MAX-CROSSING-SUBARRAY(A, low, mid, high)
 1  left-sum = −∞
 2  sum = 0
 3  for i = mid downto low
 4       sum = sum + A[i]
 5       if sum > left-sum
 6            left-sum = sum
 7            max-left = i
 8  right-sum = −∞
 9  sum = 0
10  for j = mid + 1 to high
11       sum = sum + A[j]
12       if sum > right-sum
13            right-sum = sum
14            max-right = j
15  return (max-left, max-right, left-sum + right-sum)
```



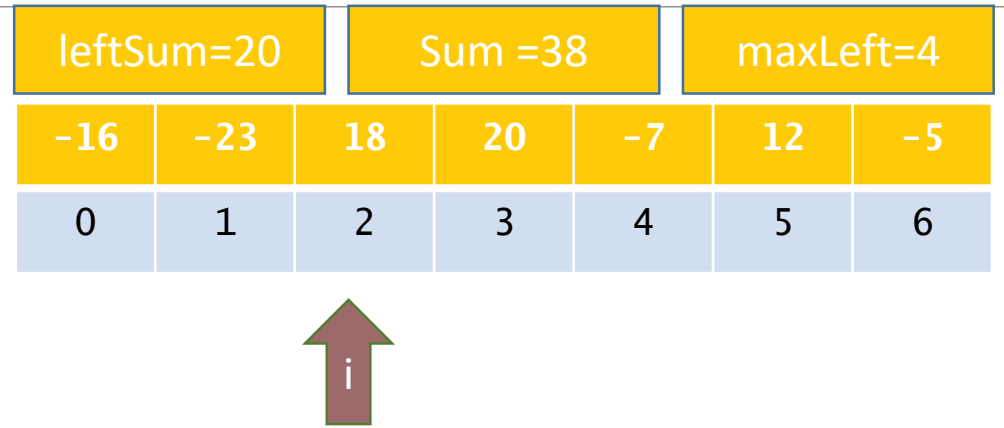
Pseudocode

```
FIND-MAX-CROSSING-SUBARRAY(A, low, mid, high)
 1  left-sum = −∞
 2  sum = 0
 3  for i = mid downto low
 4       sum = sum + A[i]
 5       if sum > left-sum
 6            left-sum = sum
 7            max-left = i
 8  right-sum = −∞
 9  sum = 0
10  for j = mid + 1 to high
11       sum = sum + A[j]
12       if sum > right-sum
13            right-sum = sum
14            max-right = j
15  return (max-left, max-right, left-sum + right-sum)
```



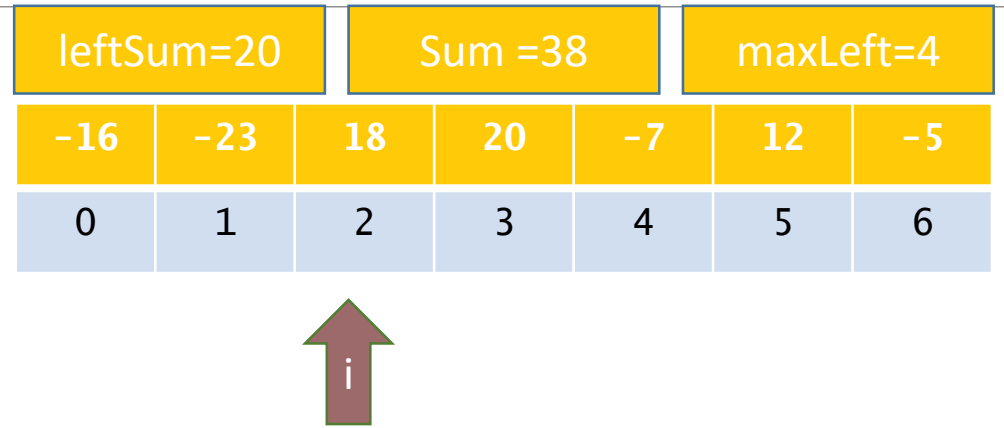
Pseudocode

```
FIND-MAX-CROSSING-SUBARRAY(A, low, mid, high)
 1  left-sum = −∞
 2  sum = 0
 3  for i = mid downto low
 4       sum = sum + A[i]
 5       if sum > left-sum
 6            left-sum = sum
 7            max-left = i
 8  right-sum = −∞
 9  sum = 0
10  for j = mid + 1 to high
11       sum = sum + A[j]
12       if sum > right-sum
13            right-sum = sum
14            max-right = j
15  return (max-left, max-right, left-sum + right-sum)
```



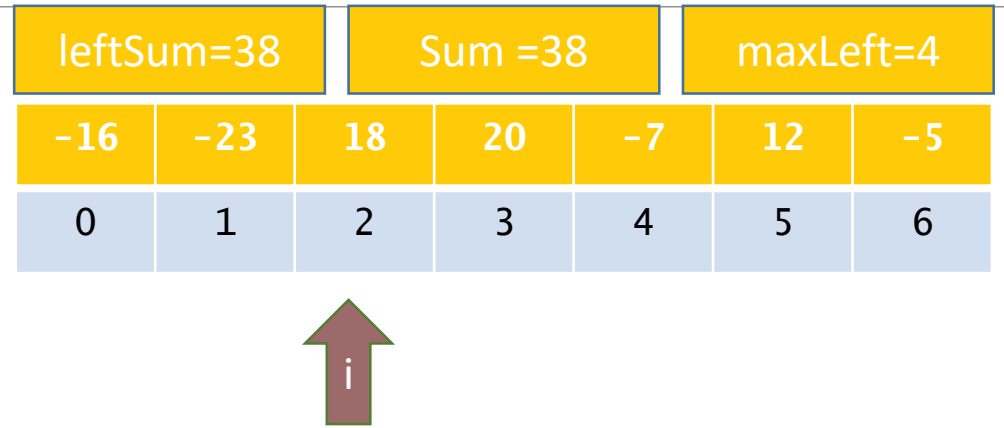
Pseudocode

```
FIND-MAX-CROSSING-SUBARRAY(A, low, mid, high)
 1  left-sum = −∞
 2  sum = 0
 3  for i = mid downto low
 4       sum = sum + A[i]
 5       if sum > left-sum
 6            left-sum = sum
 7            max-left = i
 8  right-sum = −∞
 9  sum = 0
10  for j = mid + 1 to high
11       sum = sum + A[j]
12       if sum > right-sum
13            right-sum = sum
14            max-right = j
15  return (max-left, max-right, left-sum + right-sum)
```



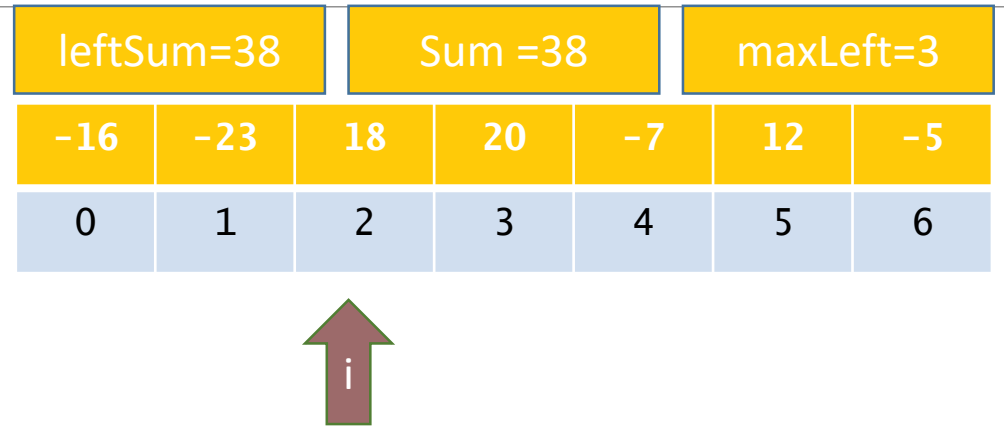
Pseudocode

```
FIND-MAX-CROSSING-SUBARRAY(A, low, mid, high)
 1  left-sum = −∞
 2  sum = 0
 3  for i = mid downto low
 4       sum = sum + A[i]
 5       if sum > left-sum
 6            left-sum = sum
 7            max-left = i
 8  right-sum = −∞
 9  sum = 0
10  for j = mid + 1 to high
11       sum = sum + A[j]
12       if sum > right-sum
13            right-sum = sum
14            max-right = j
15  return (max-left, max-right, left-sum + right-sum)
```



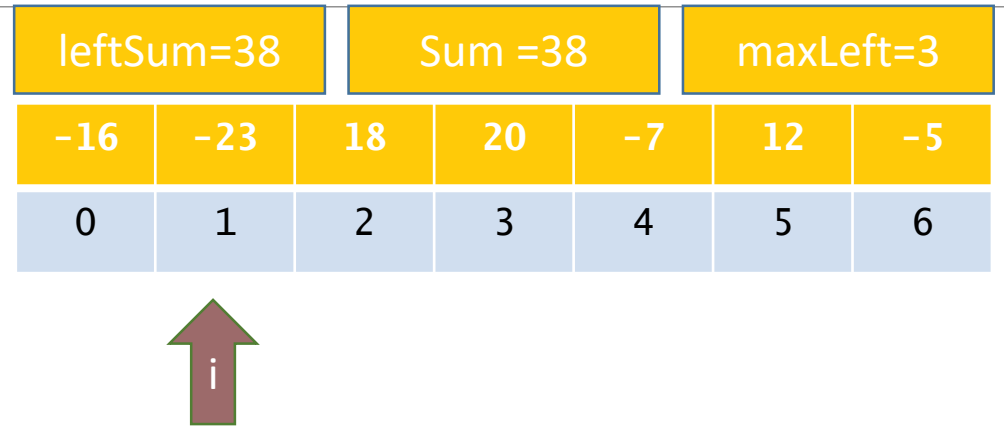
Pseudocode

```
FIND-MAX-CROSSING-SUBARRAY(A, low, mid, high)
 1  left-sum = −∞
 2  sum = 0
 3  for i = mid downto low
 4       sum = sum + A[i]
 5       if sum > left-sum
 6            left-sum = sum
 7            max-left = i
 8  right-sum = −∞
 9  sum = 0
10  for j = mid + 1 to high
11       sum = sum + A[j]
12       if sum > right-sum
13            right-sum = sum
14            max-right = j
15  return (max-left, max-right, left-sum + right-sum)
```



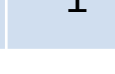
Pseudocode

```
FIND-MAX-CROSSING-SUBARRAY(A, low, mid, high)
 1  left-sum = −∞
 2  sum = 0
 3  for i = mid downto low
 4       sum = sum + A[i]
 5       if sum > left-sum
 6            left-sum = sum
 7            max-left = i
 8  right-sum = −∞
 9  sum = 0
10  for j = mid + 1 to high
11       sum = sum + A[j]
12       if sum > right-sum
13            right-sum = sum
14            max-right = j
15  return (max-left, max-right, left-sum + right-sum)
```



Pseudocode

```
FIND-MAX-CROSSING-SUBARRAY(A, low, mid, high)
 1  left-sum = −∞
 2  sum = 0
 3  for i = mid downto low
 4       sum = sum + A[i]
 5       if sum > left-sum
 6            left-sum = sum
 7            max-left = i
 8  right-sum = −∞
 9  sum = 0
10  for j = mid + 1 to high
11       sum = sum + A[j]
12       if sum > right-sum
13            right-sum = sum
14            max-right = j
15  return (max-left, max-right, left-sum + right-sum)
```



leftSum=38		Sum =15			maxLeft=3	
-16	-23	18	20	-7	12	-5
0	1	2	3	4	5	6



Pseudocode

```
FIND-MAX-CROSSING-SUBARRAY(A, low, mid, high)
 1  left-sum = −∞
 2  sum = 0
 3  for i = mid downto low
 4       sum = sum + A[i]
 5       if sum > left-sum
 6            left-sum = sum
 7            max-left = i
 8  right-sum = −∞
 9  sum = 0
10  for j = mid + 1 to high
11       sum = sum + A[j]
12       if sum > right-sum
13            right-sum = sum
14            max-right = j
15  return (max-left, max-right, left-sum + right-sum)
```

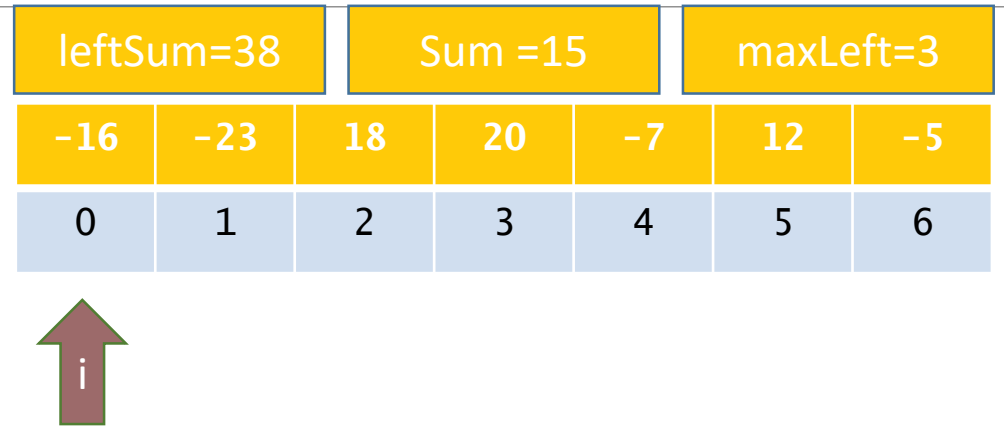


leftSum=38		Sum =15			maxLeft=3	
-16	-23	18	20	-7	12	-5
0	1	2	3	4	5	6



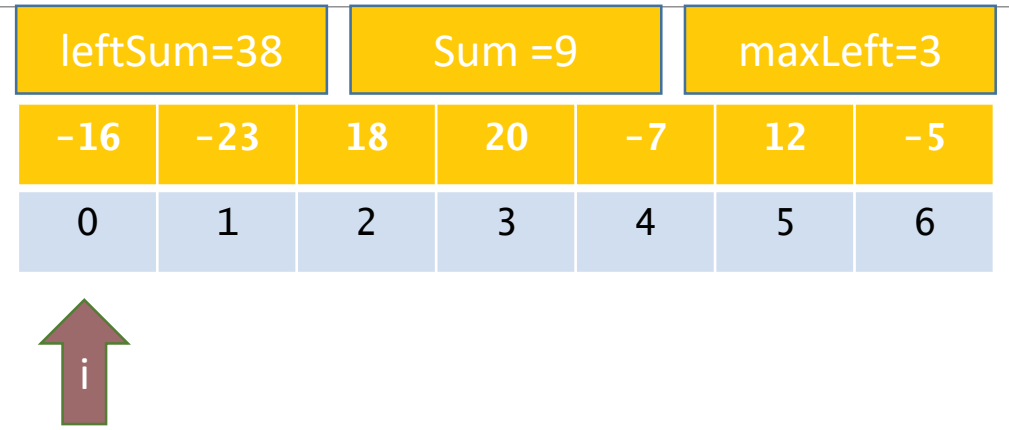
Pseudocode

```
FIND-MAX-CROSSING-SUBARRAY(A, low, mid, high)
 1  left-sum = −∞
 2  sum = 0
 3  for i = mid downto low
 4       sum = sum + A[i]
 5       if sum > left-sum
 6            left-sum = sum
 7            max-left = i
 8  right-sum = −∞
 9  sum = 0
10  for j = mid + 1 to high
11       sum = sum + A[j]
12       if sum > right-sum
13            right-sum = sum
14            max-right = j
15  return (max-left, max-right, left-sum + right-sum)
```



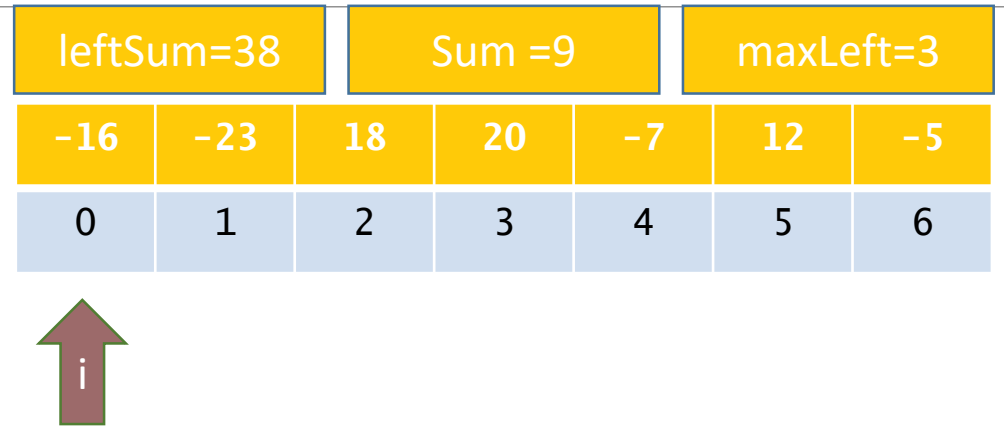
Pseudocode

```
FIND-MAX-CROSSING-SUBARRAY(A, low, mid, high)
 1  left-sum = −∞
 2  sum = 0
 3  for i = mid downto low
 4       sum = sum + A[i]
 5       if sum > left-sum
 6            left-sum = sum
 7            max-left = i
 8  right-sum = −∞
 9  sum = 0
10  for j = mid + 1 to high
11       sum = sum + A[j]
12       if sum > right-sum
13            right-sum = sum
14            max-right = j
15  return (max-left, max-right, left-sum + right-sum)
```



Pseudocode

```
FIND-MAX-CROSSING-SUBARRAY(A, low, mid, high)
 1  left-sum = −∞
 2  sum = 0
 3  for i = mid downto low
 4       sum = sum + A[i]
 5       if sum > left-sum
 6            left-sum = sum
 7            max-left = i
 8  right-sum = −∞
 9  sum = 0
10  for j = mid + 1 to high
11       sum = sum + A[j]
12       if sum > right-sum
13            right-sum = sum
14            max-right = j
15  return (max-left, max-right, left-sum + right-sum)
```



Pseudocode

```
FIND-MAX-CROSSING-SUBARRAY(A, low, mid, high)
 1  left-sum = −∞
 2  sum = 0
 3  for i = mid downto low
 4       sum = sum + A[i]
 5       if sum > left-sum
 6            left-sum = sum
 7            max-left = i
 8  right-sum = −∞
 9  sum = 0
10  for j = mid + 1 to high
11       sum = sum + A[j]
12       if sum > right-sum
13            right-sum = sum
14            max-right = j
15  return (max-left, max-right, left-sum + right-sum)
```



leftSum=38		Sum =9			maxLeft=3	
-16	-23	18	20	-7	12	-5
0	1	2	3	4	5	6

Pseudocode

```
FIND-MAX-CROSSING-SUBARRAY(A, low, mid, high)
 1  left-sum = −∞
 2  sum = 0
 3  for i = mid downto low
 4       sum = sum + A[i]
 5       if sum > left-sum
 6            left-sum = sum
 7            max-left = i
 8  right-sum = −∞
 9  sum = 0
10  for j = mid + 1 to high
11       sum = sum + A[j]
12       if sum > right-sum
13            right-sum = sum
14            max-right = j
15  return (max-left, max-right, left-sum + right-sum)
```



leftSum=38		Sum =9			maxLeft=3	
-16	-23	18	20	-7	12	-5
0	1	2	3	4	5	6

Similarly the right sum is determined

Time Complexity

FIND-MAX-CROSSING-SUBARRAY : $\Theta(n)$,

FIND-MAXIMUM-SUBARRAY

$$T(n) = \begin{cases} \Theta(1) & \text{if } n = 1, \\ 2T(n/2) + \Theta(n) & \text{if } n > 1. \end{cases}$$

$T(n) = 2T(n/2) + \Theta(n)$

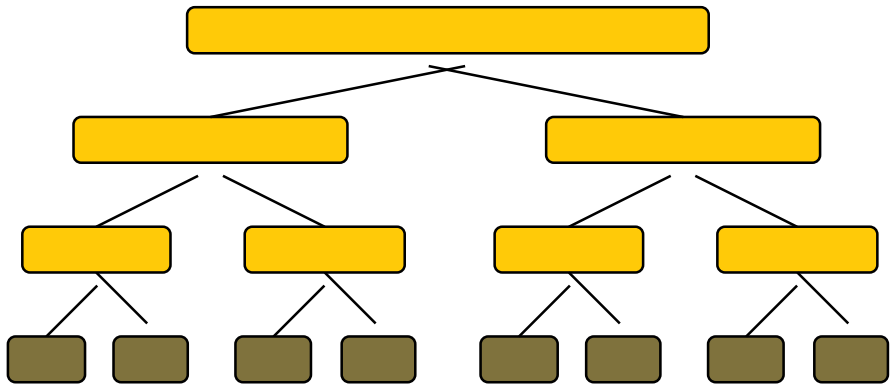
$= \Theta(n \log n)$ (similar to merge-sort)

Time Complexity (Recursive Tree)

- Draw the **recursion tree** for the recurrence relation and look for a pattern:

$$T(n) = \begin{cases} b & \text{if } n = 1 \\ 2T(n/2) + bn & \text{if } n \geq 2 \end{cases}$$

depth	T's	size	Time for max crossing sub array
0	1	***n***	***bn***
1	2	***n/2***	***bn: (bn/2+bn/2)***
…	…	…	***bn: (bn/4+bn/4+bn/4+bn/4)***
i	2^i	$n/2^i$	***…***



Total time = *i* * ***bn*** ***i = log n.*** ***T(n) = bnlog n +bn***

Time Complexity (Iterative Approach)

$$
\begin{aligned}
T(n) &= 2T(n/2) + bn \\
&= 2(2T(n/2^2)) + b(n/2)) + bn \\
&= 2^2 T(n/2^2) + 2bn \\
&= 2^3 T(n/2^3) + 3bn \\
&= 2^4 T(n/2^4) + 4bn \\
&= \ldots \\
&= 2^i T(n/2^i) + ibn
\end{aligned}
$$

- Note that base, $T(n) = b$, case occurs when $2^i = n$.
  That is, $i = \log n$.
- So, $T(n) = bn + bn \log n$

- Thus, $T(n)$ is $O(n \log n)$.

Similar Problems

You are given an array P [1..100], where an index is marked as 1 if it is a prime number or 0 if it is not a prime number (see the array below for reference). Determine the total number of prime numbers in a given array using a divide and conquer algorithm. Draw the recursive tree and write the pseudocode.

1	2	3	4	5	6	.	.	99	100
0	1	1	0	1	0			0	0

Conclusion: Divide-and-Conquer

- This Divide and conquer algorithm is clearly substantially **faster** than any of **the brute-force methods.** It required some cleverness, and the programming is a little more complicated – but the payoff is large.

- Divide and conquer is just **one of several powerful** techniques for algorithm design
- Divide-and-conquer algorithms can be **analyzed using** recurrences
- Can lead to **more efficient algorithms**

THANK YOU

